PLAIN-LANGUAGE GUIDE

How Amplifi works

What the Amplifi platform does, how its parts fit together, and where each part lives in the code. It is written so anyone on the Battery POC team can follow it and explain it.

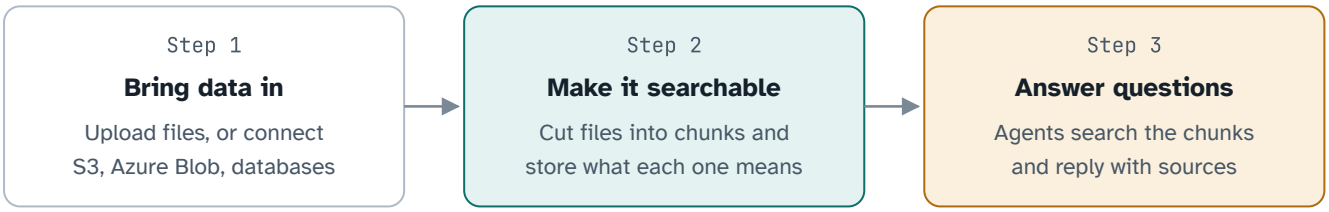
Date 26 October 2026

Based on the amplifi-dev repository: README, CLAUDE.md, the user docs in `docusaurus/docs` and the service READMEs, checked against the code

Scope read-only; nothing in Amplifi was changed

Amplifi in one sentence

Amplifi takes an organisation's documents and data, makes them searchable by meaning, and lets AI agents answer questions about them while showing the passages they used. It is built by ThoughtsWin Systems and runs on Microsoft Azure.



Three things to remember

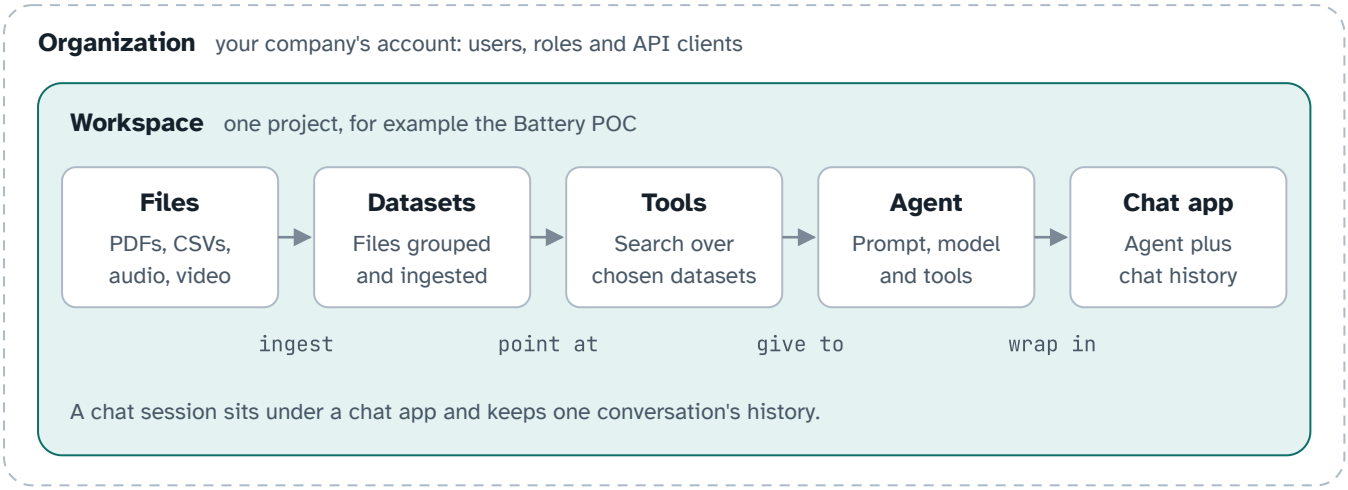
1. Everything lives inside a **workspace**. Files are grouped into **datasets**, datasets are searched by **tools**, and tools are used by **agents**.
2. Slow work, like reading 30 PDFs, is done by separate **worker services** that pick jobs off a queue. The API itself always answers quickly.
3. An agent is a **prompt + a language model + search tools**. It searches first, reads what it found, then writes the answer.

What is in this guide

2 The building blocks and their names
3 The big picture: every part on one page
4 Journey 1: getting documents in
5 Journey 2: asking a question
6 How search finds the right passage
7 Knowledge graphs, SQL and other features
8 The 12 services and their job queues
9 Logins, roles and where it runs
10 The code map, and what it means for BRP

The building blocks and their names

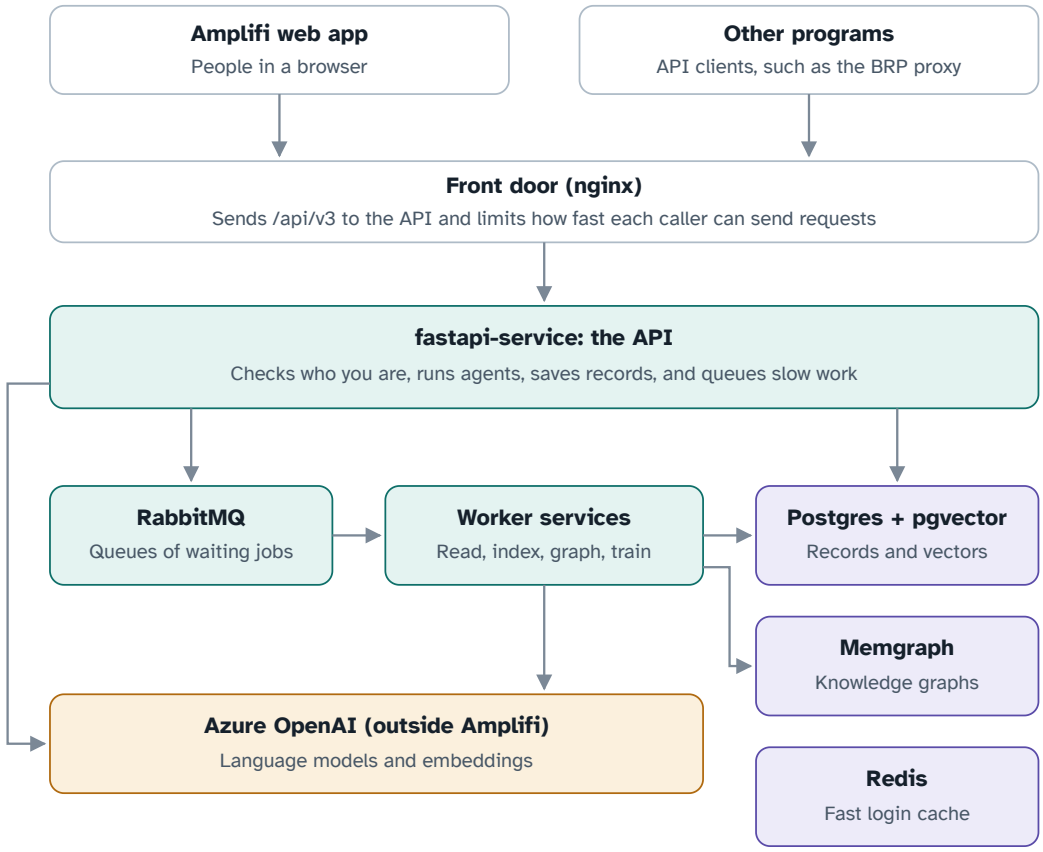
Amplifi organises everything in a strict hierarchy. Once you know these names you can read almost any screen, document or API route in the project.



Term	What it means in plain words	In the Battery POC
Organization	The company account. It holds users, their roles and API clients.	The org that owns the POC workspace
Workspace	A project space. Everything below lives inside exactly one workspace.	The workspace with Theme A and Theme B
File	Anything uploaded or pulled in: PDF, Word, PowerPoint, Excel, CSV, image, audio or video. The web app accepts up to 30 MB per file.	The 37 paper PDFs
Dataset	A named group of files that are ingested together and searched together.	"Theme A" (lithium-ion) and "Theme B-Client-Papers" (CP01–CP10)
Ingestion	Reading the files, cutting them into chunks and storing them so they can be searched.	Run once per dataset
Chunk	A short passage of a file. Search always returns chunks, never whole files.	A paragraph on a capacity test
Embedding	A list of numbers describing what a chunk means, so passages with similar meaning sit close together.	Made by Azure OpenAI
Tool	An ability an agent can use, such as Vector Search, pointed at chosen datasets.	Tool-01 File Navigator, Tool-02 Vector Search, Tool-03 Hybrid Search
Agent	A prompt (instructions), a language model and a set of tools.	The chat agent on prompt v4.7, GPT-5.2
Chat app, session	A chat app is an agent ready for chatting. A session is one conversation and its history.	Used in the Amplifi web app today
API client	A login for programs rather than people: a client ID and a secret.	What the BRP proxy will use

The big picture: every part on one page

Amplifi is not one program. It is a set of cooperating services, each running in its own container. Requests come in at the top; slow work is passed down to workers; everything ends up stored in Postgres.



fastapi-service

The only part that speaks HTTP. It checks logins, reads and writes records, runs agents for chat, and puts slow jobs on a queue.

RabbitMQ

A post office for jobs. Each kind of job has its own named queue, and the matching worker collects from it.

Worker services

Separate programs, each doing one kind of job: reading documents, transcribing audio, building graphs, training SQL models. Built on Celery.

Postgres + pgvector

The main database: users, workspaces, datasets, agents, chat history, job results, and every chunk with its embedding.

Memgraph and Redis

Memgraph stores knowledge graphs. Redis is a short-term memory, for example a login that was just checked.

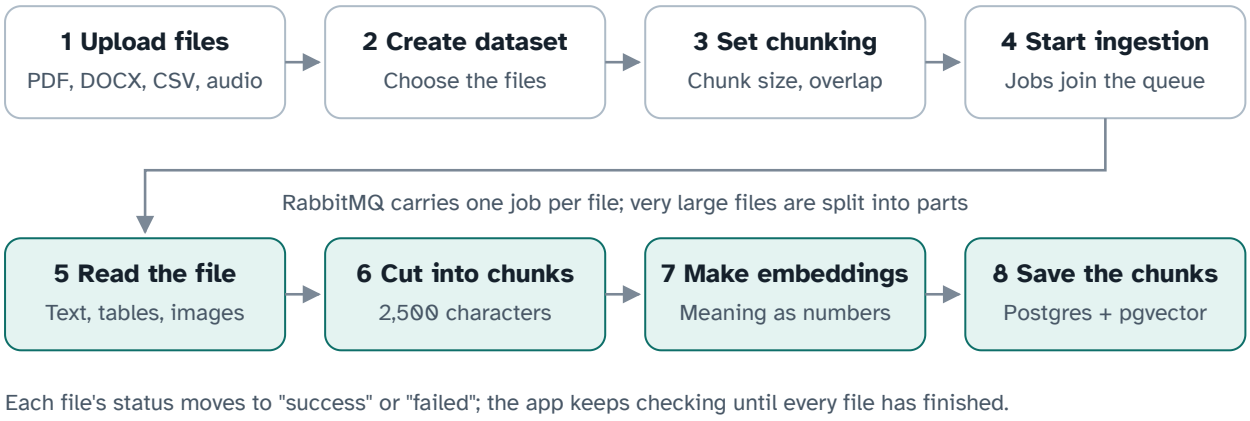
Azure OpenAI

Writes answers, makes embeddings and tidies questions. Amplifi's rules keep customer data in region.

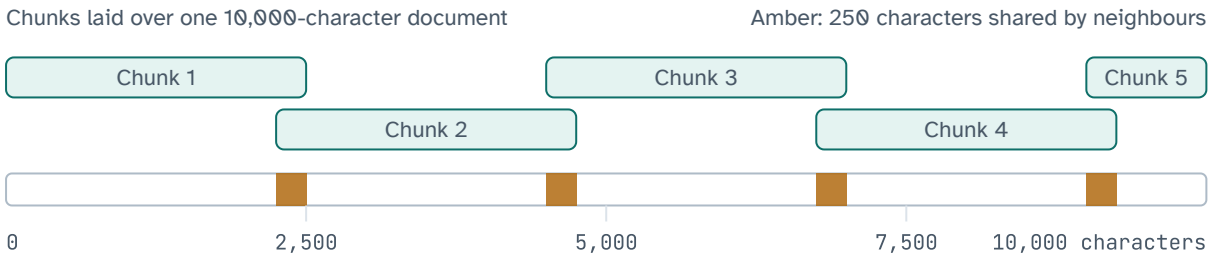
Also running: nginx allows 10 requests a second per caller; Temporal runs long workflows that must survive crashes; Vault holds secrets; Flower, Prometheus and Grafana monitor it all. **Older diagram:** `static/hld_view_Amplifi.png` shows an earlier design with an "R2R server" doing search. The current code does that itself.

Journey 1: getting documents in

Before anyone can ask a question, files must be ingested. The first four steps happen through the API, from the web app or another program. The API then hands the heavy work to a worker, so the screen does not freeze while 30 PDFs are read.



What chunking with overlap looks like

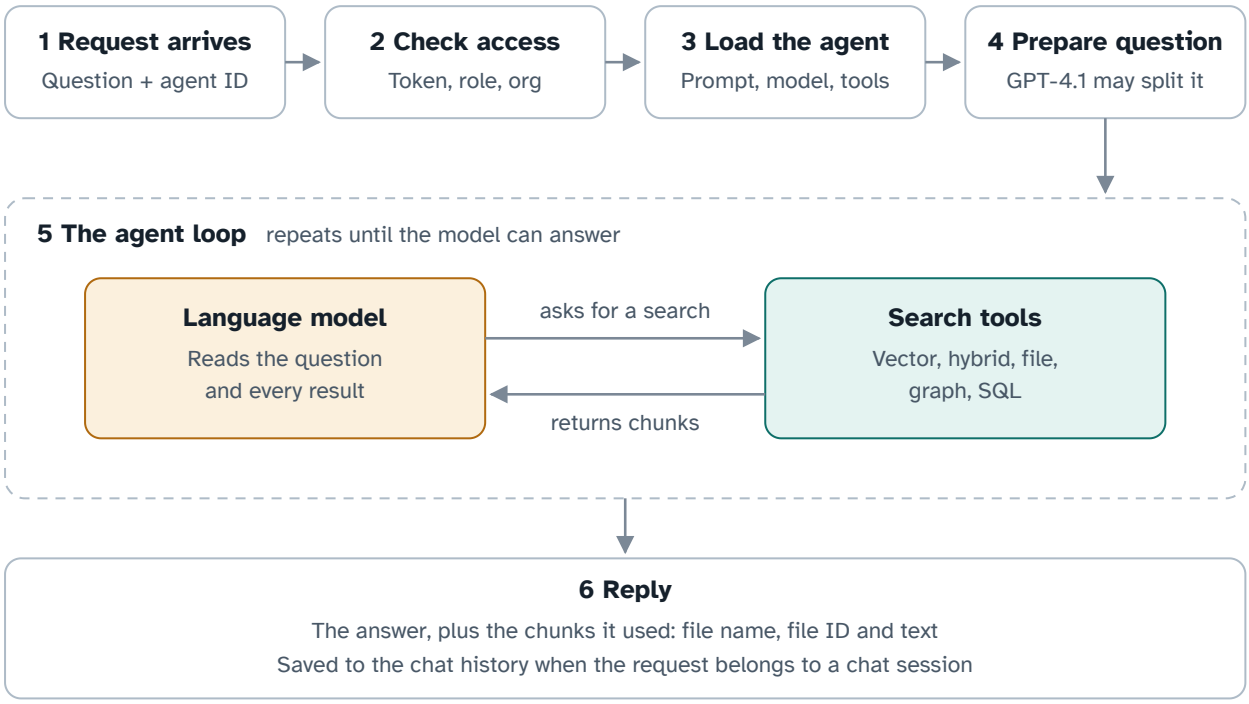


The web app's default is 2,500 characters per chunk with 250 characters of overlap, roughly half a PDF page. The overlap means a sentence cut at a boundary still appears whole in one chunk. The user docs describe the size in tokens (about 2.5 pages), but the setting the code sends is `max_characters`.

Kind of input	Worker that handles it	What the worker does
Documents	`document-ingestion-service`	Reads PDF, Word, PowerPoint, HTML, Excel, CSV and Markdown with Docling, then chunks and embeds them
Images	`image-ingestion-service`	Reads text with OCR, describes the picture with a vision model, then embeds it
Audio	`audio-ingestion-service`	Transcribes with Azure Speech, identifies speakers, then chunks the transcript
Video	`video-ingestion-service`	Extracts and transcribes the audio, samples frames, then chunks it
Database tables	`file-pull-service`	Copies tables from Postgres, MySQL, Redshift or Fabric for SQL questions. Tables are not chunked.

Journey 2: asking a question

A question goes to an agent. The agent does not answer from memory. It runs a loop: the language model decides what to search for, a tool does the search, the model reads the results, and it repeats until it has enough to answer.



Three ways to send a question

Route	Who can call it	What comes back
`POST /chat/upload`	People and API clients	One JSON reply with no history. An optional `output_schema` forces the answer into a fixed JSON shape. PDFs or images can be attached.
`POST /chat`	Signed-in people	One JSON reply, saved to a chat session so follow-up questions keep their context
`POST /chat/stream`	Signed-in people	The same reply, streamed word by word while it is written

Details that surprise people

The question may be rewritten. Before the agent starts, GPT-4.1 checks whether the message holds several questions. If it does, the agent answers each part separately and the reply has one entry per part.

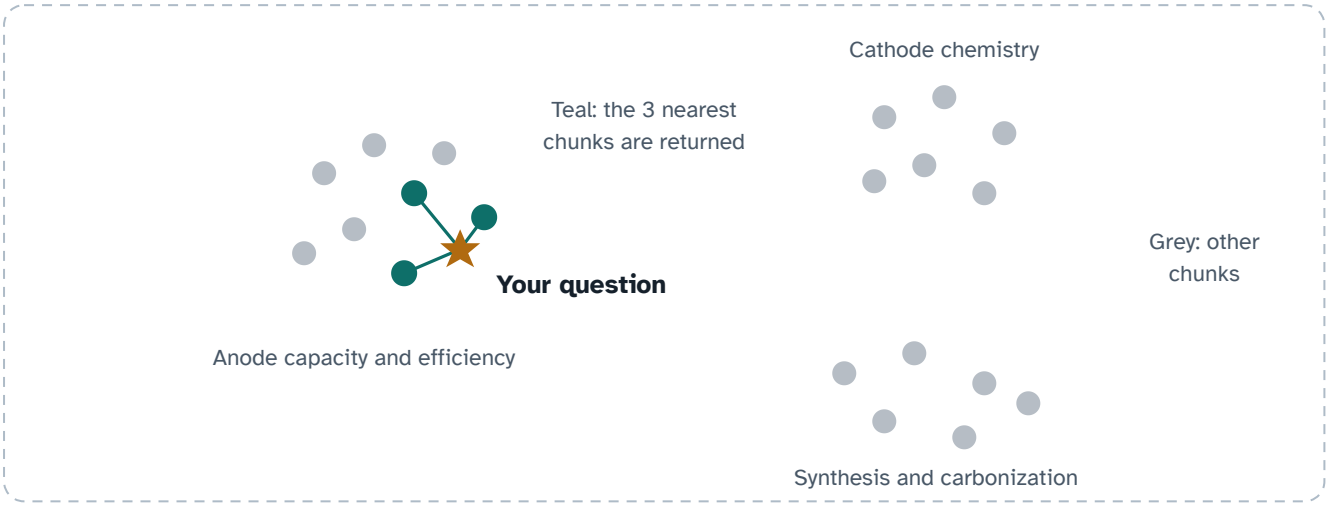
History is short. The model sees only the last 5 messages of a session.

Sources are chunks, not pages. The reply lists the chunks the tools returned. Page numbers in an answer come from the model reading the text, so they need checking.

Errors can look like success. If the agent fails, `/chat/upload` still returns HTTP `200` with an `error` field.

How search finds the right passage

Every chunk's embedding places it on a "map of meaning". Passages about similar things land close together, even when they use different words. A question is placed on the same map, and search returns the chunks nearest to it.



Example question: "What is the reversible capacity of BGP@HEC-HC?" Nearness is measured with cosine distance by default; L2 and Lp distances can be chosen on the Search page, which also reports precision, ranking quality (NDCG) and speed per dataset.

The built-in tools an agent can use

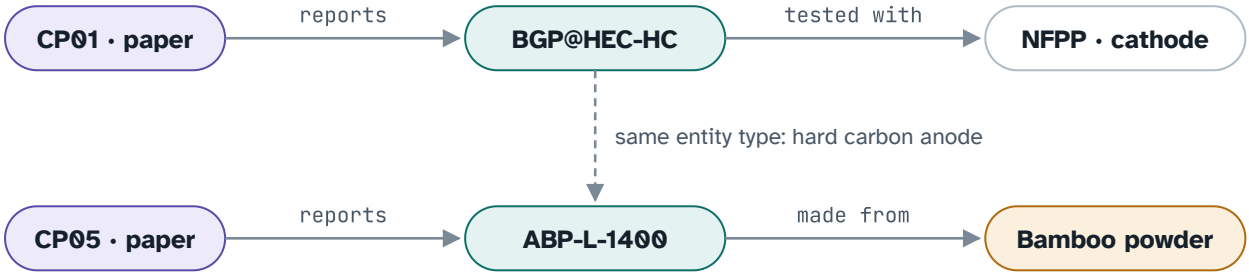
Tool	What it does	Example question
Vector Search	Finds chunks closest in meaning to the question	"Find passages on first-cycle efficiency"
Hybrid Search	Combines meaning with exact keyword matches, then merges the two rankings	Exact sample names such as "ABP-L-1400"
File System Navigator	Lists and reads files in the workspace by name	"Which file is Shen 2026?"
Text to SQL	Writes SQL for a connected database and runs it	"Average capacity by anode type"
Graph Search	Follows links in a knowledge graph	"Which cathodes were tested with this anode?"
Visualization	Draws a chart from data	"Plot capacity against efficiency"
Web Search	Searches the internet through Tavily	Current news or standards
Video Search	Searches inside indexed videos using transcripts	"Where was the roadmap discussed?"

Find the file first, then search inside it. When an agent has both File Navigator and a search tool, Amplifi tells it to locate the file before searching. The Theme B tools follow that order: Tool-01 File Navigator, Tool-02 Vector Search, then Tool-03 Hybrid Search.

Knowledge graphs, SQL and other features

Knowledge graphs: facts as connected dots

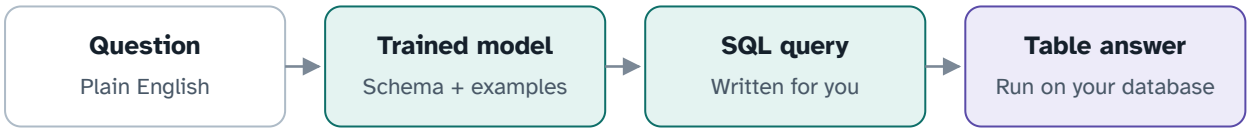
A knowledge graph is built from an ingested dataset in two passes: first the **entities** (materials, papers, people, organisations), then the **relationships** between them. `graph-service` does the work and stores the result in Memgraph. Graph Search then lets an agent follow the links, which suits questions about connections.



Illustrative example drawn from the POC's papers; it is not an actual graph from Amplifi.

Text to SQL: questions about tables

For data that lives in a database, ingestion does not chunk anything. Instead `training-service` trains a Vanna model on the table layout, some written notes and example question-and-SQL pairs. The Text to SQL tool then turns a plain question into SQL, runs it and returns a table.



Data sources

Files can be uploaded directly, or pulled from Azure Blob Storage, AWS S3, Groove tickets and databases (Postgres, MySQL, Redshift, Microsoft Fabric). `monitoring-service` checks Azure and Groove for new files on a timetable kept by `celery-beat-service`.

Destinations and workflows

A workflow copies a dataset's vectors to another store, such as PG Vector or Databricks Vector Search, daily, weekly or monthly.

MCP tools

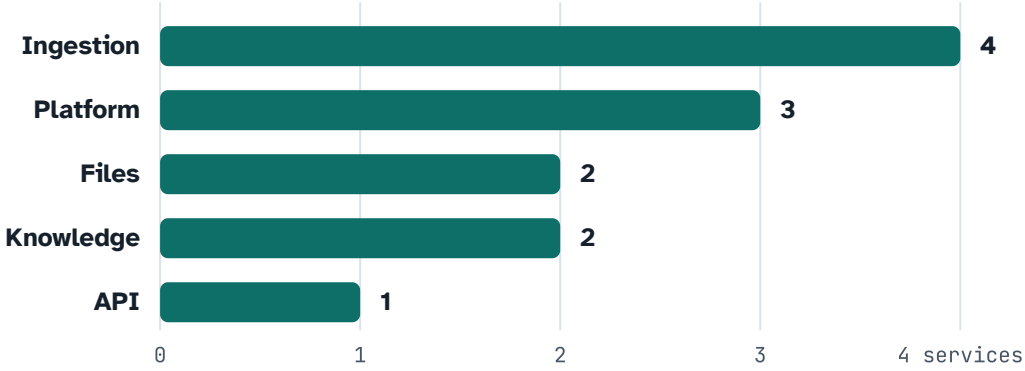
MCP (Model Context Protocol) lets an agent use tools that live outside Amplifi, such as a company search API. Tavily web search comes built in.

Extraction

A newer feature that pulls structured fields out of documents. It runs on Temporal in `extraction-worker`, so a crash resumes where it stopped. A setting switches it on.

The 12 services and their job queues

The backend in `backend_new/services/` is 12 separate programs. Only one of them, the API, talks to the outside world. The other 11 wait for work.

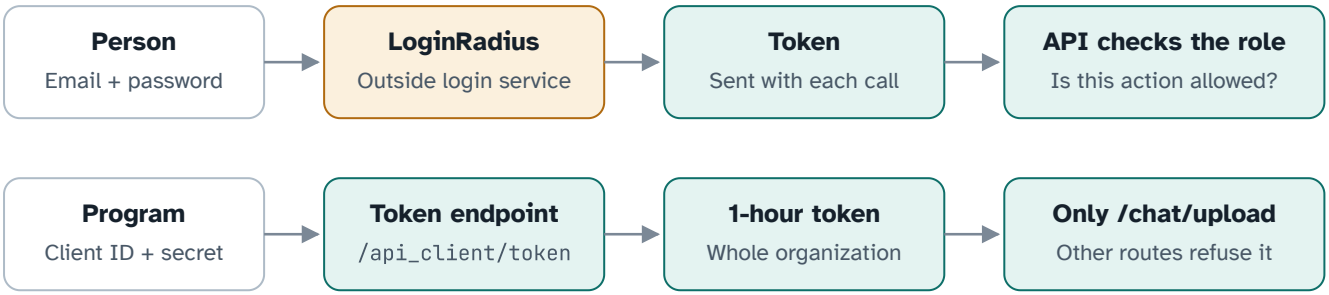


Service	Its job	Takes work from
`fastapi-service`	The API for everything; puts jobs on queues	HTTP requests
`document-ingestion-service`	Documents into chunks and embeddings	`ingestion_queue_v3`
`image-ingestion-service`	OCR and vision for images	`image_ingestion_queue_v3`
`audio-ingestion-service`	Transcripts and speakers	`audio_ingestion_queue_v3`
`video-ingestion-service`	Audio and frames from video	`video_ingestion_queue_v3`
`file-pull-service`	Pulls from S3, Azure, Groove and databases	`file_pull_queue_v3`, plus one queue per database type
`file-compression-service`	Checks and shrinks PDFs	`file_compression_queue_v3`, `check_pdf_queue_v3`
`graph-service`	Entities and relationships into Memgraph	`build_graph_queue_v3`
`training-service`	Trains Text-to-SQL (Vanna) models	`vanna_training_queue_v3`
`celery-beat-service`	The clock: schedules repeating jobs	Its own timetable, stored in Postgres
`monitoring-service`	Watches for new files and clears stuck documents	Jobs scheduled by celery-beat
`extraction-worker`	Runs the extraction workflows	Temporal

How one job travels. The API never does slow work itself. It writes a job onto a named queue in RabbitMQ and replies at once. The worker for that queue picks the job up, does the work and writes the result to Postgres. The app checks the status later. Locally, the Flower page at `localhost:5555` shows every queue and worker.

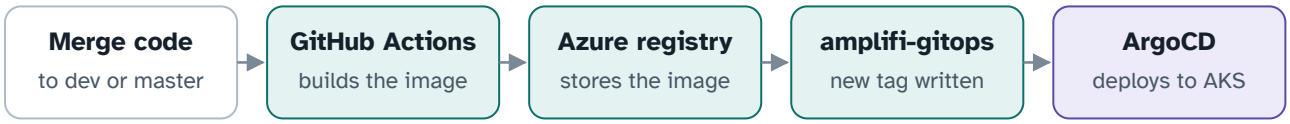
Logins, roles and where it runs

Two ways to log in



Role	What it can do
Admin	Everything in the organization: users, settings and every workspace, without being added to each one
Developer	Build things: workspaces, datasets, ingestion, tools, agents and API clients. Most create and change routes need Admin or Developer.
Member	Use what already exists: search, chat, and view datasets and agents

How a change reaches the servers



Environment	API address	Notes
Local	`http://localhost:8085`	Docker Compose with about 23 containers (`new-docker-compose-dev.yml`)
Dev	`dev-api.dataamplifi.com`	Deployed on every merge
Stage	`stage-api.dataamplifi.com`	Checked before a release
Prod	Set per deployment	Released from a version tag (vX.Y.Z) on master. Separate ONGC and Azure Marketplace deployments also exist.

In the cloud everything runs on Azure Kubernetes Service, behind an Azure Application Gateway, with Azure PostgreSQL, Key Vault for secrets, Container Registry and Azure Monitor (`static/container_architecture.png`). Every API address ends in `/api/v3`.

The code map, and what it means for BRP

What is in the repository

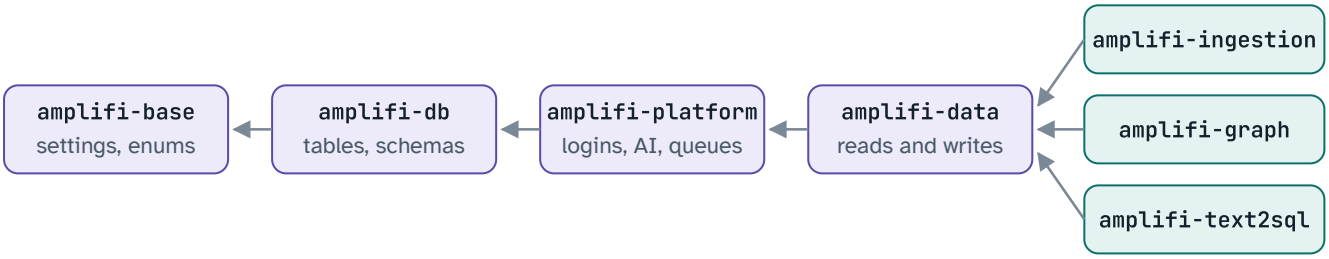
```
amplifi-dev/
├─ backend_new/       the live backend
│  ├─ services/       12 programs
│  ├─ packages/       shared code
│  └─ tests/e2e/      API walkthrough
├─ frontend/          web app (Next.js)
├─ docusaurus/docs/   user docs
├─ backend/           old, frozen: ignore
├─ nginx/ k8s/        routing, deploys
└─ static/            diagrams (some old)
```

How the backend code is written

Python 3.12, with **uv** managing packages. Each service is a container image. Services never import each other; they share code only through `packages/`, all imported under one name, `shared.*`.

Settings come from environment variables in a `.env` file, which holds real secrets and is never committed.

The shared packages, from bottom layer to top



Arrows mean "uses". A package may only use the layers to its left. Newer packages for extraction, page layout and vision sit alongside these.

To understand...	Open
Every API route	`backend_new/services/fastapi-service/fastapi_service/api/`
The chat routes	`…/api/v2/endpoints/chat.py`
The agent loop	`…/fastapi_service/utils/dynamic_agent.py`
The search tools	`…/fastapi_service/tools/system_tools/`
Database tables	`backend_new/packages/amplifi-db/src/shared/models/`
All settings	`backend_new/packages/amplifi-base/src/shared/config.py`
A full API walkthrough	`backend_new/tests/e2e/workflows/shared_workflow_steps.py`

What this means for the Battery POC

- BRP does not change Amplifi. It calls one route, `POST /api/v3/chat/upload`, through a small server-side proxy that holds the API client secret.
- Papers, datasets, tools and agents keep being managed in the Amplifi web app.
- `output_schema` can force the JSON shapes BRP's screens need.
- Open points, such as question splitting, page numbers and timeouts, are listed in `docs/amplifi-api-notes.md`.